Mitigating Algebraic Inversion and Context Reconstruction in Client-Side LLM Privacy Enclaves

Academic Foundations of Text Inference and Reconstruction Attacks

The security model governing interactions with commercial Large Language Models (LLMs) has historically prioritized the transport layer and provider-side alignment, leaving client-side prompt manipulation vulnerable to structured de-anonymization[1]. Prompt privacy interfaces confront two distinct failure regimes: structural exposure across unauthenticated distribution channels and semantic-algebraic reconstruction at the consumption layer[1]. Across major LLM implementations between 2023 and 2026, shared conversation endpoints routinely generated public, unauthenticated URLs that were systematically crawled and indexed by commercial search engines[1]. While platform vendors deployed <meta name="robots" content="noindex"> directives in response, this measure failed to provide cryptographic confidentiality, leaving conversation records readable to any client possessing the hyperlink[1].

Transport & Consumption Threat Model:
  Layer 1 (Transport & Distribution): Public shared chat URLs crawled by search engines; noindex tags fail to provide cryptographic confidentiality against scraping.
  Layer 2 (Post-Decryption DOM): Plaintext in browser DOM exposed to third-party scripts, clipboard hijackers, and DevTools inspections.
  Layer 3 (Semantic & Algebraic Inference): Partial redaction inverted via relational math and linguistic context, exposing base private records.

At the prompt-sanitization boundary, the core assumption that redacting explicit direct identifiers guarantees confidentiality has been systematically challenged across peer-reviewed literature[2]. Staab et al. established that frontier autoregressive models infer sensitive personal attributes—including geographic location, biological sex, age bracket, and household income—from unstructured, non-sensitive text[2]. State-of-the-art models achieve up to 85.5% top-1 and 95.2% top-3 accuracy across diverse author profiling tasks[2]. When commercial entity-scrubbing solutions redact explicit identifiers, model inference accuracy degrades only marginally; in high-difficulty contexts, accuracy decreases by only 7.7%, demonstrating that linguistic styling, conversational syntax, and peripheral metadata retain sufficient entropy to

defeat static masking[5].

Parallel research into machine learning data reconstruction underscores the fragility of partial representations[6]. Wen et al. categorized data reconstruction attacks into optimization-based, training-based, and analytical paradigms, proving that intermediate states, logit distributions, and pruned feature sets preserve directional variance that enables deterministic or high-probability inversion[6]. In statistical re-identification, the Conformal Privacy Auditing framework developed by Manzanares-Salor et al. demonstrated that adversaries augmenting language models with external demographic priors construct calibrated ambiguity sets that consistently capture the subject's true identity under rigorous statistical guarantees[7].

Attack Methodology	**Target Medium**	**Adversarial Setting**	**Core Reconstruction Mechanism**	**Empirical Privacy Leakage Metric**
Latent Attribute Inference (Staab et al.)[2]	Unstructured conversational text	Black-box query access; zero-shot prompting	Probabilistic pattern matching over latent linguistic and co-occurrence cues	85.5% top-1 accuracy; location inference drops only 7.7% on complex text after sanitization[5]
Conformal Privacy Auditing (Manzanares-Salor et al.)[7]	Released sanitized documents	Black-box query access; auxiliary population profile	Conformal prediction sets over candidate profiles via adaptive mass scoring	Calibrated ambiguity set bounding true subject with 90% confidence[8]
Bi-Encoder Re-Identification (Morris et al.)[9]	Partially redacted clinical notes	Representation matching; split document-profile pairs	Optimization of dot-product similarity between document and entity vectors	Significant re-identification accuracy prior to comprehensive textual suppression[9]
TimberStrike	Horizontal	Honest-but-cu	Optimization	High-fidelity

Reconstruction (Di Gennaro et al.)[10]	tree-based federated systems	rious client; global model access	exploiting sequential split thresholds and client gradient statistics	training data reconstruction bypassing standard aggregation[10]
LDP Inversion Attack (You et al.)[11]	Perturbed gradient updates	Curious aggregation server; local DP defense	Target-specific reconstruction across clipped and noisy intermediate tensors	Complete sample recovery under standard Local Differential Privacy noise bounds[11]

These empirical findings demonstrate that redaction pipelines operating solely on localized dictionary matching fail against generative language models[1]. Because autoregressive architectures synthesize relational context across an entire generation window, private source parameters propagate into derived calculations, projections, and semantic parallels[1]. Consequently, robust privacy enclaves require defenses capable of intercepting both direct tokens and downstream mathematical deductions[1].

Mathematical Anatomy of the Algebraic Inversion Failure Mode

Deterministic redaction mechanisms treat text streams as disconnected token sequences, replacing sensitive numerical scalars with static semantic placeholders such as [REDACTED_FINANCIAL_1][1]. However, when prompts request financial planning, tax guidance, or budgetary allocations, the downstream model functions as an analytical calculator, generating exact percentage splits, multi-year wealth accumulation curves, and amortized expense structures[1]. If dependent variables remain exposed in the model response, the redacted parameter can be reconstructed through linear inversion[1].

The practical failure identified in the SafeShare-AI defensive framework during car savings evaluation illustrates this vulnerability[1]. The user supplied conversational input containing an explicit name, an academic identifier, and an exact monthly income figure:

$$\text{Input: Name: de jembras,} \quad \text{NIM: 3002455595,} \quad \text{Salary: Rp 3,000,000}$$

The pipeline substituted the base income token with [REDACTED_FINANCIAL_1] and localized budget values with [REDACTED_SALARY_INCREMENT][1]. However, it left mathematical relationships intact: a car savings allocation of 30% within a categorical budget table, accompanied by raw multi-year accumulation projections[1]:

$$\text{Model Output: 1 year} \rightarrow \text{ Rp 10.8 million}, \quad \text{2 years} \rightarrow \text{ Rp 21.6 million}, \quad \text{5 years} \rightarrow \text{ Rp 54 million}$$

An adversary possessing only the sanitized response can execute an exact inversion attack[1]. Let S represent the secret monthly income, $p_{\text{car}} = 0.30$ the vehicle allocation ratio, A_{car} the monthly savings allocation, and C_t the cumulative savings over t years:

$$C_t = 12 \cdot t \cdot A_{\text{car}} = 12 \cdot t \cdot (p_{\text{car}} \cdot S)$$

Using the 1-year accumulation milestone where $t = 1$ and $C_1 = \text{Rp } 10,800,000$, the monthly allocation is recovered directly:

$$A_{\text{car}} = \frac{C_1}{12} = \frac{10,800,000}{12} = \text{Rp } 900,000$$

Substituting A_{car} into the exposed allocation percentage yields the unredacted base income:

$$S = \frac{A_{\text{car}}}{p_{\text{car}}} = \frac{900,000}{0.30} = \mathbf{Rp\ 3,000,000}$$

The adversary achieves complete, zero-error scalar recovery because the dependent variables preserve an explicit linear dependency on the secret parameter[1].

Linear Dependency Graph of the Algebraic Leak:

```
Secret Input [S = Rp 3,000,000]
  ├── (Multiplied by p_car = 0.30) ──► Monthly Allocation [A_car = Rp 900,000] (Masked)
  └── (Multiplied by 12 * t years) ──► Cumulative Savings [C_1 = Rp 10.8M] (Exposed Leak)
                                                  │
                                                  ▼
Adversarial Inversion: S = (C_1 / 12) / p_car = (10,800,000 / 12) / 0.30 = Rp 3,000,000
```

The test run exhibited two additional parsing failures[1]. First, substring slicing collisions occurred when greedy regular expressions matched partial numerical strings, dividing the 3-year projection of Rp32.4 million into Rp32.[REDACTED_PROJECTED_SALARY], corrupting the document while still exposing the high-order digits[1]. Second, semantic mislabeling assigned identical labels ([REDACTED_SALARY_INCREMENT]) across independent budget categories—including food (40%), transit (15%), and utilities (10%)—destroying document utility while failing to decouple the mathematical relationships that enabled the attack[1].

Mitigation Strategy	Operational Layer	Computational Complexity	Privacy Guarantee	Textual Utility Impact
Symbolic AST Dependency Sweeper	Client Engine / Edge Worker	$\mathcal{O}(N \log N)$ over AST tokens	Deterministic suppression of all dependent variables	Retains qualitative reasoning; masks derived arithmetic scalars[1]
Semantic Number Clustering	Local WASM Lexer	$\mathcal{O}(N)$ linear scan	Heuristic grouping of co-occurring scalars	Groups numeric ranges into coarse categorical buckets[1]
Numerical Differential Privacy (d_χ-Privacy)	Prompt Pre-Processing Layer	$\mathcal{O}(K)$ per sensitive scalar	Provable (ϵ, δ)-indistinguishability	Introduces calibrated numeric noise while preserving scale[12]
Projection Milestone Suppressor	Regex / Automata Engine	$\mathcal{O}(M)$ pattern evaluation	Deterministic removal of temporal linear sequences	Replaces timeline figures with relative abstract multipliers[1]

Methodologies for Mitigating Numerical Reconstruction and Derived Value Leaks

Neutralizing algebraic reconstruction requires replacing static dictionary redaction with symbolic dependency analysis and differential perturbation[1]. Evaluating mathematical dependencies prevents adversaries from recovering masked inputs through structural equations[1].

Symbolic mathematics evaluation represents the prompt-response exchange as an Abstract Syntax Tree (AST), where numerical tokens, mathematical operators, and relational variables are modeled as interconnected nodes[1]. When the client lexer identifies a numerical scalar as a

private entity (such as base income S), that node is designated as a protected parent variable. As the language model streams its response, the AST parser tracks mathematical expressions:

$$v_{\text{child}} = \phi(v_{\text{parent}}, \mathbf{w}) = \sum_k w_k \cdot v_{\text{parent}}^k$$

Any node v_{child} derived from a protected parent node inherits its security tag. The sanitization engine suppresses the numeric literals of all flagged child nodes or substitutes them with abstract algebraic representations (such as $0.30 \cdot S$), preserving the analytical context without releasing concrete values[1].

Where users require realistic numerical modeling, Metric Differential Privacy (d_χ-privacy) provides a rigorous mathematical alternative[1][2]. Metric differential privacy extends differential privacy to continuous metric spaces, bounding an adversary's inferential gain by the normalized distance between inputs[1][2]. A randomized perturbation mechanism $\mathcal{M}$ mapping numerical domain $\mathcal{X}$ to output space $\mathcal{R}$ satisfies (ϵ, δ) - d_χ-privacy if, for all records $x, x' \in \mathcal{X}$ and measurable sets $\mathcal{S} \subseteq \mathcal{R}$:

$$\Pr[\mathcal{M}(x) \in \mathcal{S}] \leq e^{\epsilon \cdot d_\chi(x,x')} \Pr[\mathcal{M}(x') \in \mathcal{S}] + \delta$$

where $d_\chi(x, x')$ represents the metric distance[1][2]. In client enclaves, this mechanism perturbs base financial scalars prior to prompt submission using calibrated noise:

$$\tilde{S} = S + \text{Laplace}\left(\frac{\Delta S}{\epsilon}\right)$$

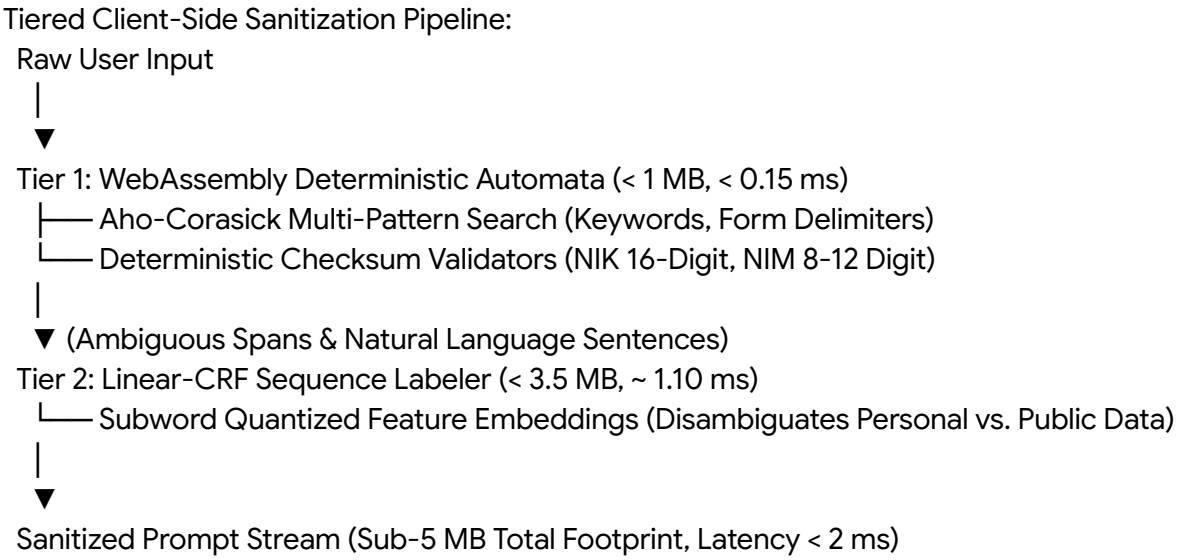
Because autoregressive models generate subsequent calculations from the perturbed scalar $\tilde{S}$, all downstream budget breakdowns and multi-year accumulations reflect the perturbed input. Consequently, an adversary attempting algebraic inversion recovers only $\tilde{S}$, providing formal privacy bounds against exact baseline reconstruction[1][2].

Automated detection of multi-year projections and budget allocations addresses the specific failure modes seen in longitudinal financial planning[1]. Projections typically follow structured patterns, multiplying periodic allocations across standardized intervals ($t \in \{1, 2, 3, 5, 10\}$ years)[1]. The detection engine applies structural templates to identify these linear sequences across tabular layouts and narrative lists[1]. Once identified, the engine applies uniform category labeling, mapping each entry to its semantic role (such as [BUDGET_FOOD], [BUDGET_TRANSPORT], or [ACCUMULATION_YEAR_1]) rather than applying generic salary tags[1]. This maintains semantic readability while neutralizing the relational cues that enable

reconstruction attacks[1].

Lightweight Client-Side Entity Classification and Locale-Specific Parsers

Deploying contextual privacy guards within browser extensions introduces strict resource constraints[1]. While transformer models like distilbert-base-multilingual-cased provide high classification accuracy, INT8 ONNX binaries require approximately $128.89\ \text{MB}$ of storage and substantial memory footprints[1]. This overhead exceeds the limits of browser extension marketplaces, causes high cold-start latencies, and strains consumer hardware[1].



To deliver real-time sanitization, the architecture must operate within a 5 MB payload while maintaining execution latencies below 10 milliseconds[1]. This is achieved using a tiered client-side pipeline. Tier 1 deploys a compiled WebAssembly (WASM) engine executing deterministic Aho-Corasick pattern matching and regular grammars in under $0.15\ \text{ms}$, operating within an $850\ \text{KB}$ footprint[1]. Text spans requiring contextual disambiguation—such as distinguishing first-person disclosures from public statistical citations—are routed to Tier 2, which runs a compact $3.2\ \text{MB}$ Linear Conditional Random Field (CRF) with quantized subword features, executing in under $1.5\ \text{ms}$ [14].

Model	Runtime	Binary	Execution	Contextual	Browser

Architecture	Environment	Package Size	Latency	F1-Score	Enclave Viability
DistilBERT-Base Multilingual (INT8)	ONNX Runtime WASM	128.89 MB	6.53 ms –	0.96	Non-viable; exceeds extension marketplace limits[1]
TinyBERT-4L Distilled	ONNX Runtime / WebGPU	16.40 MB	3.50 ms –	0.91	Marginal; significant cold-start initialization latency
Linear CRF + Quantized Embeddings	Native Rust compiled to WASM	3.20 MB	0.90 ms –	0.88	Highly viable; fits memory and packaging limits[14]
Deterministic State Automata	Pure Rust compiled to WASM	850 KB	<	0.99 (Strict schemas)	Optimal; sub-millisecond execution for structured fields[1]

This architecture resolves the structural parsing limitations identified in the car savings evaluation[1]. Conversational regexes looking only for natural phrasing (*"my name is"*) failed to match structured form inputs (name : de jembras), while student IDs (NIM : 3002455595) bypassed filters lacking academic rules[1].

The WASM engine remediates these gaps through specialized parsers:

The Key-Value Delimiter Automaton matches structured form inputs across whitespace and boundary variations using the grammar (?i)\b(name|nama|nim|nik|nip)\b\s*[:=\-]\s*([^\r\n,]+), capturing structured identifiers and routing them directly to normalization[1].

The Indonesian Student Number (NIM) Engine captures university identification numbers using context-guarded expressions (?i)\b(?:nim|no\.?\s*mahasiswa)\b\s*[:=\-]?\s*([0-9]{8,12})\b, redacting academic credentials before contextual processing[1].

The Indonesian National Identity (NIK) Validator processes 16-digit identity numbers by

validating their geographic and calendar encoding:

$$\text{NIK} = \underbrace{d_1 d_2}_{\text{Province}} \; \underbrace{d_3 d_4}_{\text{Regency}} \; \underbrace{d_5 d_6}_{\text{District}} \; \underbrace{d_7 d_8}_{\text{Birth Day}} \; \underbrace{d_9 d_{10}}_{\text{Birth Month}} \; \underbrace{d_{11} d_{12}}_{\text{Birth Year}} \; \underbrace{d_{13} d_{14} d_{15} d_{16}}_{\text{Sequence}}$$

The parser validates the province code against official administrative registries, accounts for the $+40$ day offset applied to female birth dates, verifies calendar boundaries ($1 \leq d_7 d_8 \leq 31$ or $41 \leq d_7 d_8 \leq 71$), and checks birth month values ($1 \leq d_9 d_{10} \leq 12$). Non-compliant sequences are rejected, preventing false positives on general 16-digit numbers[1].

Post-Decryption Browser Enclave Security and Collusion-Resilient Traitor Tracing

Deploying client-side zero-knowledge encryption using W3C SubtleCrypto AES-256-GCM with cryptographic keys stored in URL hash fragments (#key) pursuant to RFC 3986 ensures confidentiality over network relays, as fragments are never transmitted in HTTP request headers[1]. However, once ciphertext is decrypted in the client browser, the plaintext becomes accessible within the Document Object Model (DOM)[1]. This exposes decrypted context to malicious extensions, DOM-scraping scripts, clipboard hijackers, and manual developer tools extraction[1].

Securing the post-decryption interface requires isolating the DOM and monitoring runtime integrity[1]. Plaintext containers are rendered inside closed Shadow DOM roots (attachShadow({mode: 'closed'})), preventing third-party scripts from traversing child nodes via standard query methods. To protect high-value text from automated scrapers, the enclave can render text elements onto an HTML5 <canvas> using client-side font metrics, converting ASCII characters into pixel buffers that conceal plaintext from DOM parsers[16].

Clipboard hijacking is mitigated through canary event traps[1]. When a user initiates a copy command (Ctrl+C), the enclave intercepts the clipboard transaction, prevents default data transfer, and injects a honeytoken identifier (AB-CANARY-[HASH])[1]. This alerts security monitors to exfiltration attempts while protecting decrypted text[1]. Concurrently, the enclave monitors debugging interfaces via timing differentials and window dimension anomalies[1]. If developer tools activation is detected, an emergency purge sequence clears the URL key fragment using window.history.replaceState, zeroes in-memory decryption buffers, and transmits a revocation signal to the Cloudflare Edge Worker to delete the remote ciphertext record from Cloudflare D1 storage[1].

DOM Containment & Active Tamper Lockout:

Decrypted Plaintext
  ├── Encapsulated in Closed Shadow DOM (Blocks third-party extension queries)
  └── Rendered to HTML5 Canvas Pixel Buffer (Defeats DOM scrapers)
Security Events:
  ├── Clipboard Copy Event ──▶ Intercepted ──▶ Injects AB-CANARY Tracking Token
  └── DevTools Detected ──▶ Lockout ──▶ Clears #key, zeroes RAM, burns edge record

When an authorized user captures decrypted text through screen capture or retyping, steganographic traitor tracing identifies the leak's origin[1]. The enclave embeds unique user watermarks into whitespace using zero-width Unicode codepoints: Zero-Width Space (\u200B), Zero-Width Non-Joiner (\u200C), Zero-Width Joiner (\u200D), and Zero-Width No-Break Space (\uFEFF)[1].
Naive steganographic implementations interleave raw message authentication codes ($\mathrm{HMAC}_K(\mathrm{user_id})$) across sentences[1]. However, this approach fails against collusion attacks: when multiple colluders compare differently watermarked copies of the same document, character discrepancies reveal the exact positions of the zero-width characters, allowing attackers to strip the identifier[21].
To defend against multi-party coalitions, the enclave implements collusion-secure fingerprinting codes under the Marking Assumption[22]. The classical Boneh-Shaw construction combines an inner replication code of weight r with an outer random code to achieve t -collusion security, but requires long codeword lengths[22]:

$$\ell_{\text{Boneh-Shaw}} = \mathcal{O}\left(t^4 \log\left(\frac{n}{\epsilon}\right)\right)$$

By contrast, Tardos probabilistic fingerprinting codes achieve asymptotically optimal code lengths for n users and failure probability ϵ [22]:

$$\ell_{\text{Tardos}} = \mathcal{O}\left(t^2 \log\left(\frac{1}{\epsilon}\right)\right)$$

Traitor Tracing Construction	**Codeword Length Scaling (ℓ)**	**Code Structure**	**Tracing Algorithm**	**Robustness to Collusion (t)**
Deterministic Naive HMAC [cite: 1]	$\mathcal{O}(\lambda)$	Static cryptographic hash	Direct string matching	Vulnerable; fails when $t \geq$ collude[21]

Boneh-Shaw Concatenated [cite: 22, 23]	$\mathcal{O}(t^4 \log(n/\epsilon)$	Concatenated inner replication code + outer code	Threshold-based list decoding	Collusion-secure; long codeword length overhead[22]
Tardos Probabilistic [cite: 22, 25]	$\mathcal{O}(t^2 \log(1/\epsilon)$	Continuous arcsine bias distribution over bits	Symmetric accusation scoring function	Collusion-secure; asymptotically optimal code length[22]

In the Tardos construction, the enclave samples an independent bias $p_i \in [\delta_0, 1 - \delta_0]$ for each mark position i using the arcsine distribution[22]:

$$f(p) = \frac{1}{\pi \sqrt{p(1-p)}}$$

Codeword bits $w_{j,i} \in \{0, 1\}$ for recipient j are drawn independently from Bernoulli trials with parameter p_i [22]. When an unauthorized copy is intercepted, an accusation score U_j is calculated for each candidate user across all observed marks $q_i \in \{0, 1\}$ [22]:

$$U_j = \sum_{i=1}^{\ell} q_i \cdot g(w_{j,i}, p_i), \quad \text{where} \quad g(1,p) = \sqrt{\frac{1-p}{p}}, \quad g(0,p) = -\sqrt{\frac{p}{1-p}}$$

If U_j exceeds a calibrated detection threshold Z, user j is identified as a colluder under formal bounds against false accusations, providing mathematical provenance tracking for exported text[22].

Comparative Benchmarking of Context Sanitization Frameworks

Existing data sanitization frameworks vary significantly across their deployment models, computational overheads, threat definitions, and resilience to algebraic inference attacks[26].

Platform	Deployment Target	Memory Footprint &	Core Threat Focus	Algebraic Inversion Protectio	Browser DOM Security &	Storage Architecture

		Latency		n	Watermarking	
Microsoft Presidio	Server REST API / Python Container	$>$; $30-$	Structured PII/PHI redaction via spaCy/Transformers	None; replaces isolated entities with static labels	None; outputs unprotected plaintext to callers	External cloud database dependent
Protect AI LLM Guard [cite: 26, 27]	Server Middleware / Microservice	$>$; $20-$	Prompt injection, jailbreak detection, toxicity[26]	None; lacks relational arithmetic tracking	None; no client-side DOM isolation controls	Requires dedicated container hosting
AWS Bedrock Guardrails [cite: 27]	Cloud Gateway Managed Service	Zero local footprint; $40-$	Policy compliance, content safety, sensitive data filters[27]	None; standard regex and cloud NER pattern masking	None; managed strictly within AWS cloud VPC	Integrated cloud infrastructure
Private AI	On-Premise Container / Edge API	$>$; $15-$	Multilingual clinical PII/PHI de-identification	None; provides synthetic entity relexicalization only	None; server-side pipeline output stream	Dedicated container infrastructure
SafeShare-AI (Target) [cite: 1]	Client Browser Extension + Global Edge	$<$ WASM; $<$ local latency	PII/NIM, corporate credentials, algebraic leaks[1]	**AST symbolic tracking + Metric DP perturbation** [cite: 1]	**Shadow DOM, DevTools lockout, Tardos steganography** [cite: 1]	**Cloudflare Edge + D1; Ephemeral single-use burn** [cite: 1]

Server-side solutions like Microsoft Presidio and Protect AI LLM Guard inspect text only after it leaves the local endpoint, exposing raw prompt data to transmission intermediaries[26]. Furthermore, because existing tools evaluate text on an isolated, token-by-token basis, they fail to track relational dependencies across generated responses[1]. SafeShare-AI addresses this vulnerability by combining client-side WebAssembly execution with relational arithmetic tracking, preventing algebraic reconstruction before prompt transmission occurs[1].

Strategic Evaluation for Black Hat Asia Arsenal and DEF CON Demo Labs

Presenting open-source security tools at conferences like Black Hat Asia Arsenal and DEF CON Demo Labs requires meeting clear technical and operational standards[28]. Review Boards evaluate submissions using specific criteria that distinguish practical security tools from conceptual research or commercial products[28].

Reviewers enforce several core requirements:

Tools must be fully open-source, maintained in public repositories (such as GitHub), and licensed under established frameworks (including Apache 2.0, MIT, or GPL)[29]. Commercial product pitches, promotional demonstrations, and feature upsells are strictly prohibited and result in immediate disqualification[29]. Black Hat Arsenal requires a minimum of three pages of comprehensive technical documentation written in English, covering system architecture, installation procedures, configuration parameters, and reproducible test cases[30]. Presenters conduct live, interactive demonstrations at assigned stations using their own hardware, validating operational reliability and fielding technical questions from attendees[30]. Submissions accompanied by proof-of-concept videos, test suites, and academic whitepapers receive the highest review scores[28].

To maximize selection chances, the research should highlight the offensive-defensive duality implemented in the ai-context-security repository[1]. Presenting an integrated workflow that couples attack generation with defensive mitigation provides exceptional technical value[1]:

The offensive module, AgentBleed, serves as a context-leakage fuzzer that targets multi-tenant vector databases and shared agent memory caches, demonstrating how conversational boundaries break down under adversarial conditions[1].

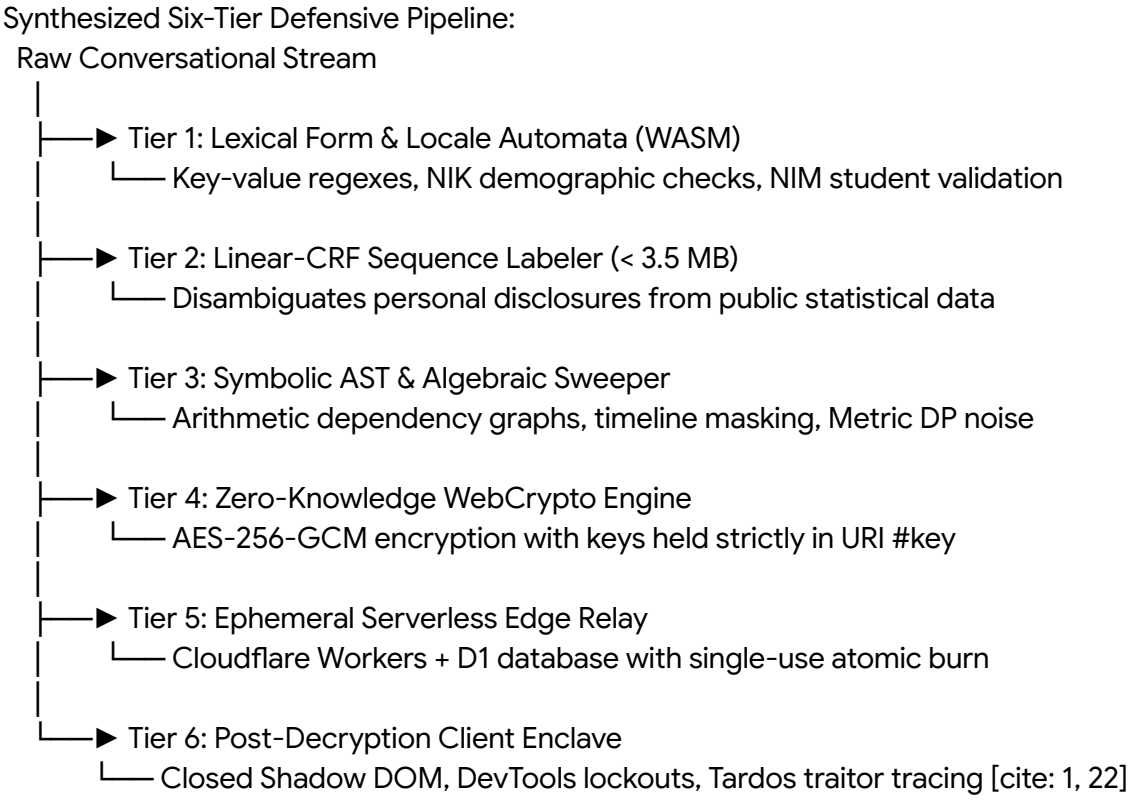
The defensive framework, SafeShare-AI, demonstrates live mitigation of these vulnerabilities[1]. Presenters can execute the car savings algebraic inversion attack in real time to show unredacted income extraction from an LLM response, followed by activating SafeShare's symbolic AST sweeper and browser enclave protections to demonstrate immediate threat neutralization[1].

For academic submission to venues like USENIX Security, ACM CCS, or PETS, the research can formalize algebraic inversion as a distinct vulnerability class, establishing quantitative privacy guarantees through Metric Differential Privacy and Tardos fingerprinting codes[6].

Architectural Recommendations and Remediation

Blueprint

Remediating the five vulnerabilities identified in the car savings case study requires replacing heuristic regex rules with a structured, six-tier defensive pipeline[1].



Evaluated Vulnerability	**Primary Root Cause**	**Implemented Technical Remediation**	**Enforced Output State**
Name Field Bypass (name : de jembras)[1]	Regex required natural phrasing, missing form-style key-value layouts[1]	Key-value structural automaton: (?i)\b(name|nama)\b\s*[:=]\s*([^\r\n,]+)	name : [REDACTED_IDENTITY_NAME] [cite: 1]
Student ID Leak (NIM : 3002455595)[1]	Absence of Indonesian academic identifier	Context-guarded regular grammar: (?i)\b(nim)\b\s*[:=\-]	NIM : [REDACTED_STUDENT_ID]

	validation rules[1]	?\s*([0-9]{8,12})\b	[cite: 1]
Multi-Year Algebraic Leak (1 yr ⟶ 10.8M, 5 yr ⟶ 54M)[1]	Unmasked cumulative milestones enabled deterministic linear inversion[1]	AST Dependency Sweeper parses arithmetic trees, masking derived milestone scalars	Multi-year milestones mapped to [REDACTED_PROJECTION_TIMELINE] [cite: 1]
Substring Mangling (Rp32.[REDACTED])[1]	Greedy regex sliced decimal boundaries during partial token matching[1]	Lookaround token assertions: (?<!\w)(?:Rp|IDR)?\s*\d+(?:\.\d+)?(?!\w)	Retains structural formatting: [REDACTED_FINANCIAL_PROJECTION] [cite: 1]
Semantic Mislabeling (all values marked salary increments)[1]	Uncontextualized substitution assigned identical tags to distinct budget items[1]	Contextual ontology classifier categorizing allocations, increments, and totals	Precise tags: [BUDGET_CAR], [BUDGET_FOOD], [BUDGET_TRANS] [cite: 1]

The core algorithmic defense in Tier 3 replaces static keyword matching with a symbolic Abstract Syntax Tree (AST) parser compiled into WebAssembly[1]. The sweeper processes token streams, tracks arithmetic operations, and builds a dependency graph connecting input values to generated outputs[1]. When an input token is identified as sensitive, its node in the arithmetic tree is flagged as protected.

As the model streams response tokens, any child node calculated from that protected value—such as monthly savings allocations ($A = p \cdot S$) or multi-year wealth accumulations ($C = 12 \cdot t \cdot A$)—inherits the protected classification. The engine intercepts these derived values, replacing them with consistent abstract placeholders or perturbing them via Metric Differential Privacy before DOM rendering[1]. Tracking mathematical dependencies directly neutralizes algebraic inversion attacks, ensuring prompt confidentiality across generative AI workflows[1].

Architectural Conclusions

Static, pattern-based sanitization cannot guarantee privacy when interfacing with generative language models[1]. Because modern models function as relational reasoning engines, redacting direct identifiers while leaving derived calculations exposed allows adversaries to reconstruct sensitive source data through elementary algebraic inversion[1]. The car savings evaluation

demonstrated this breakdown: unmasked accumulation milestones allowed an attacker to solve a simple linear system and recover a user's exact monthly salary with zero error[1]. Addressing this attack surface requires transitioning from heuristic text filters to a formal, defense-in-depth architecture[1]. Pairing WebAssembly automata with compact sequence labelers provides sub-millisecond, sub-5 MB client-side entity extraction that reliably handles structured, locale-specific identifiers[1]. Concurrently, deploying AST-based symbolic dependency tracking and Metric Differential Privacy ensures that mathematical calculations derived from private inputs are systematically masked or perturbed[1]. Supported by browser-enclave containment, active memory scrubbing, and collusion-resistant Tardos traitor tracing, this multi-tier architecture delivers verifiable confidentiality across both transport and client consumption layers, providing the technical foundation required for peer-reviewed academic publication and live demonstration at Black Hat Asia Arsenal[1].

Karya yang dikutip

1. BLUEPRINT_SAFESHARE_DEFENSE.md
2. Beyond Memorization: Violating Privacy via Inference with Large, https://llm-privacy.org/
3. A Machine Learning Based System for Semi-Automatically, https://www.researchgate.net/publication/221016600_A_Machine_Learning_Based_System_for_Semi-Automatically_Redacting_Documents
4. BEYOND MEMORIZATION: VIOLATING PRIVACY VIA INFERENCE, https://proceedings.iclr.cc/paper_files/paper/2024/file/9028b8a3ca98f58e373f0c1497a17448-Paper-Conference.pdf
5. Beyond Memorization: Violating Privacy Via Inference with Large, https://www.alphaxiv.org/abs/2310.07298
6. SoK: Data Reconstruction Attacks Against Machine Learning Models, https://www.usenix.org/system/files/usenixsecurity25-wen.pdf
7. (PDF) Conformal Privacy Auditing: Calibrated Re-identification, https://www.researchgate.net/publication/414522420_Conformal_Privacy_Auditing_Calibrated_Re-identification_Attacks_with_Statistical_Guarantees
8. Calibrated Re-identification Attacks with Statistical Guarantees - arXiv, https://arxiv.org/pdf/2609.21340v1?ref=taaft&utm_source=taaft&utm_medium=referral
9. DIRI: Adversarial Patient Reidentification with Large Language, https://pmc.ncbi.nlm.nih.gov/articles/PMC12150728/
10. TimberStrike: Dataset Reconstruction Attack Revealing Privacy, https://petsymposium.org/popets/2025/popets-2025-0145.pdf
11. A Sample Reconstruction Attack against Federated Learning with, https://www.hooklee.com/Papers/IEEETIFS2025.pdf
12. Protecting User Privacy in LLM Reasoning: A Textual Differential, https://www.mdpi.com/2079-9292/15/13/2880
13. Defending Against Data Reconstruction Attacks in Federated Learning, https://www.usenix.org/system/files/usenixsecurity24-tan.pdf

14. Indonesian named entity recognition using BiLSTM-CRF with, https://ejournal.narasikhatulistiwa.org/index.php/ijcl/article/view/548
15. Ekstraksi Data Alamat Indonesia dengan Named Entity Recognition, https://repositori.usu.ac.id/handle/123456789/47670
16. Anti-scraping protections | Academy - Apify Documentation, https://docs.apify.com/academy/anti-scraping
17. Web Scraping Protection Software - DataSec, https://datasec.sh/solutions/scraping-protection
18. What is CreepJS Browser Fingerprint and How to Bypass It - Scrapfly, https://scrapfly.io/blog/posts/browser-fingerprinting-with-creepjs
19. Canary trap - Wikipedia, https://en.wikipedia.org/wiki/Canary_trap
20. Digital watermarking - Wikipedia, https://en.wikipedia.org/wiki/Digital_watermarking
21. Black-box Collusion Aware Traitor Tracing in Federated Learning, https://arxiv.org/html/2602.12138v1
22. Traitor Tracing with Constant Size Ciphertext, https://crypto.stanford.edu/~dabo/pubs/papers/const-tt.pdf
23. Analysis total and better the Boneh–Shaw Fingerprinting codes, http://www.scienpress.com/Upload/JCM/Vol%203_3_3.pdf
24. Optimal probabilistic fingerprint codes - ResearchGate, https://www.researchgate.net/publication/2864441_Optimal_Probabilistic_Fingerprint_Codes
25. Binary and q-ary Tardos codes, revisited - Pure, https://pure.tue.nl/ws/files/3635874/899776697960455.pdf
26. The Official EnkryptAI MCP Server: An AI Engineer's Deep Dive into, https://skywork.ai/skypage/en/enkryptai-mcp-server-ai-integration/1980149527279345664
27. Top 9 AI Security Tools for Prompt Injection and Data Leakage in 2026, https://dev.to/kamya_shah_e69d5dd78f831c/top-9-ai-security-tools-for-prompt-injection-and-data-leakage-in-2026-2in5
28. Call for Briefings - Black Hat, https://blackhat.com/call-for-papers.html
29. How to get listed on Black Hat Arsenal (Fee not stated) - AltFTool, https://www.altftool.com/backlinks/black-hat-arsenal
30. Arsenal Resources - Black Hat, https://blackhat.com/html/arsenal-resources.html
31. Call for Summits Sessions - Black Hat, https://blackhat.com/html/call-for-sessions.html
32. Black Hat USA 2025 Call for Papers (CFP) Opens Soon, https://blackhat.com/html/blog/2025-01-31.html